CSE340: Computer Architecture

Handout_Chapter - 3: Arithmetic for Computers



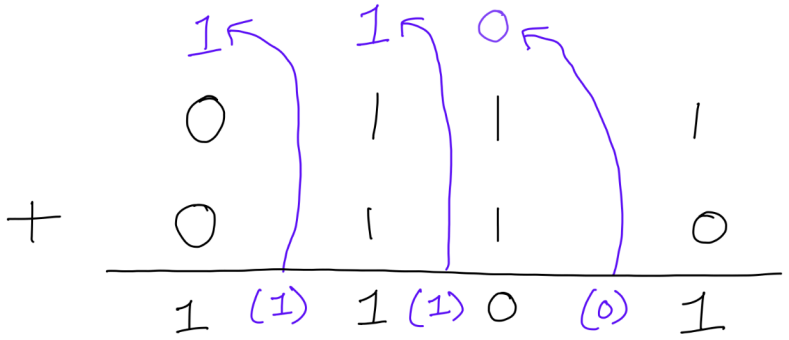
Inspiring Excellence

Prepared by: Partha Bhoumik (PBK)
Course Coordinator, CSE340

Integer Addition-Subtraction

Addition: $(7)_{10} + (6)_{10}$

Bits with this color represent carry-forward



adding bits right to left.

Subtraction: $(7)_{10} - (6)_{10} = (7)_{10} + (-6)_{10}$

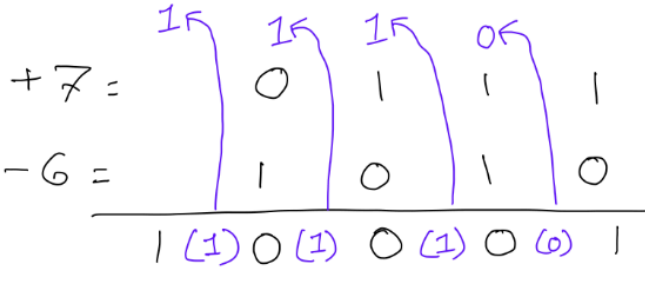
$6 = 110$

$+6 = 0110$

$= 1001$

$+1$

$-6 = (1010)_{2s}$



Overflow Detection (integer)

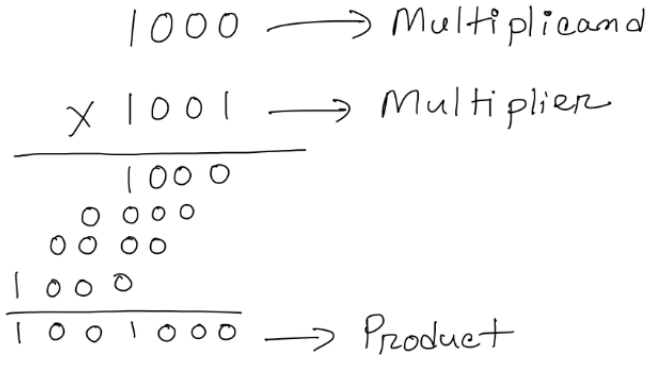
#Addition:

Case-1: Add two same signed numbers:
if (answer also has same sign):
    No overflow
else:
    Overflow
Case-2: Add two different signed numbers:
    Never Overflow

#Subtraction:

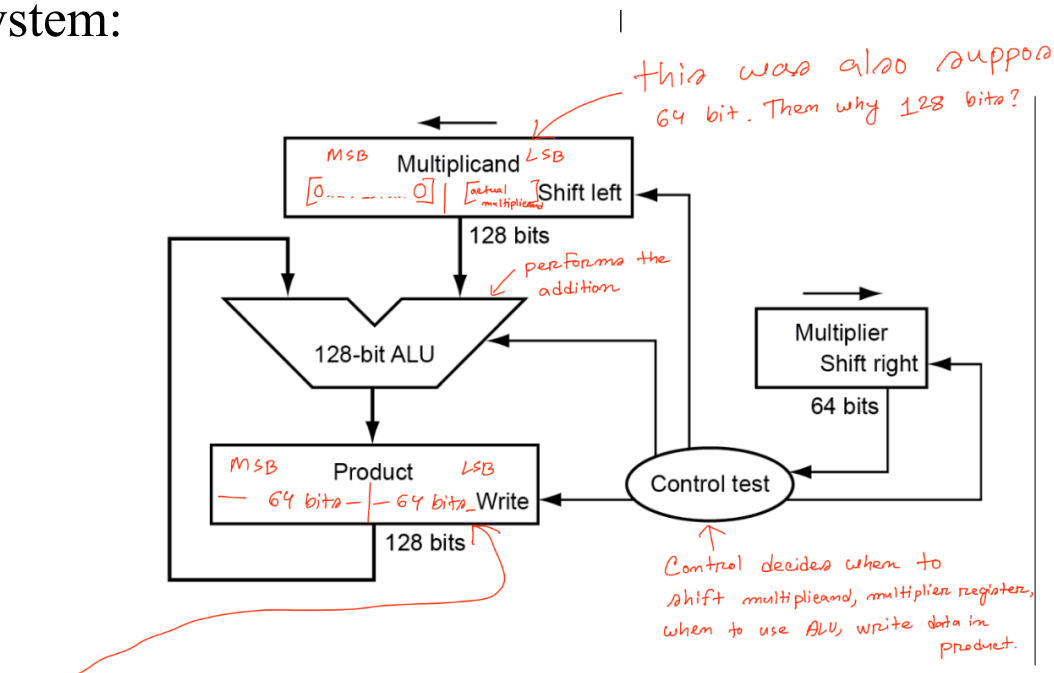
Case-3: Sub two same signed numbers:
   Never Overflow
Case-4: Sub two different signed numbers:
    +A - (-B) = +A +B => Case-1
     -A - (+B) = -A +(-B) => Case-1

Long Multiplication



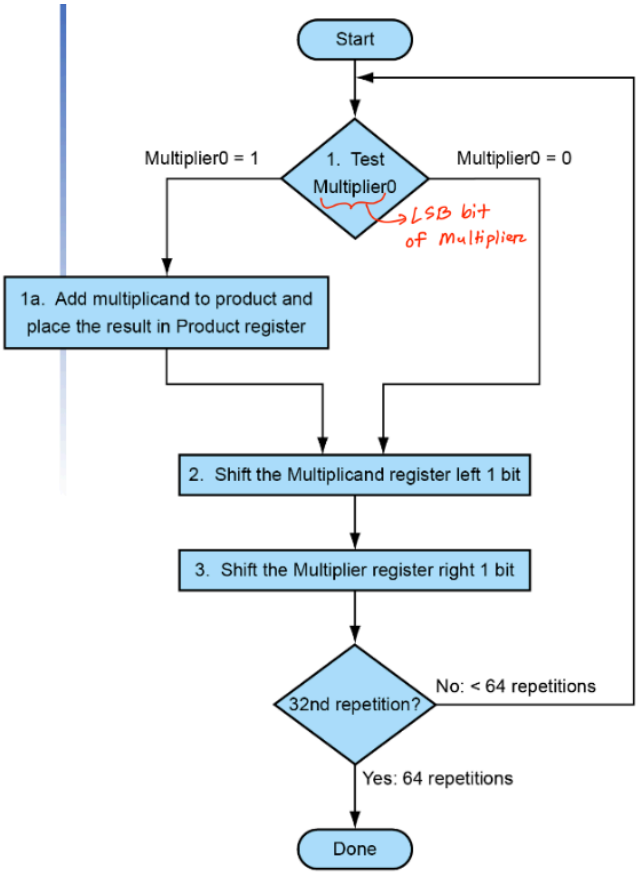
Maximum,
length of the product
= (length of multiplicand
+ " " multiplier)

System:



Multiply two 64 bit values, product length can be (64+64)=128 bit.
But we do not have any 128 bit registers in RISC-V
Hence, we use two registers to store the 0 values

Flowchart:



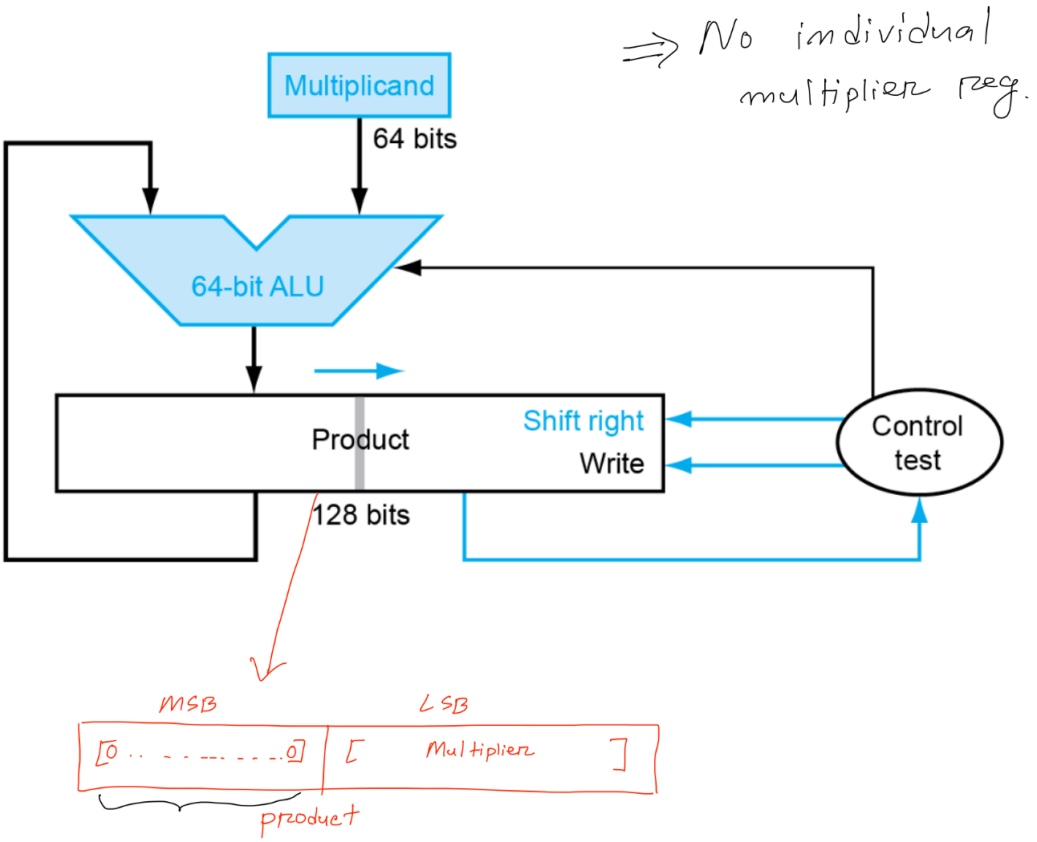
Example:

Multiply 8 and 9 using the long multiplication method:
Solution:
You can choose any of the operands as multiplier or multiplicand.
Number of iterations = Number of bits in Multiplier

Iteration	Multiplier	Multiplicand	Product
0	1001	0000 1000	0000 0000
1	1001	0000 1000	0000 1000
	1001	0001 0000	0000 1000
	0100	0001 0000	0000 1000
2	0100	0010 0000	0000 1000
	0010	0010 0000	0000 1000
3	0010	0100 0000	0000 1000
	0001	0100 0000	0000 1000
4	0001	0100 0000	0100 1000
	0001	1000 0000	0100 1000
	0000	1000 0000	0100 1000

Optimized Multiplication

System:



Logic:

```
if (iteration <= multiplier bit length):
    if (multiplier_0 == 1):
        product_MSB = Multiplicand + product_MSB
        product = right shift product by 1
    elif (multiplier_0 == 0):
        product = right shift product by 1
```

Example:

Multiply 8 and 9 using the optimized multiplication method:
Solution:
You can choose any of the operands as multiplier or multiplicand.
Number of iterations = Number of bits in Multiplier
This color represents the product MSB part.

Iteration	Multiplicand	Product
0	1000	0000 1001
1	1000	1000 1001
		0100 0100
2	1000	0010 0010
3	1000	0001 0001
4	1000	1001 0001
		0100 1000

Floating Point

How does RISC-V support numbers with fractions?
=> Using IEEE-754 floating point representation

Scientific Notation is just a way to represent very large or very small number.

$\Rightarrow 4500000 = 4.5 \times 10^{6}$

$\Rightarrow 0.00453 = 4.53 \times 10^{-3}$

Coefficient, Base, Exponent

Decimal

- $\Rightarrow 5.64 \times 10^{33}$
- $\Rightarrow -2.34 \times 10^{56}$

→ Normalized.

- $\Rightarrow 109.64 \times 10^{33}$
- $\Rightarrow 0.002 \times 10^{-4}$
- $\Rightarrow +987.02 \times 10^{9}$

→ Not Normalized.

In Binary,

$\pm 1.xxxx_2 \times 2^{yyy}$

IEEE-754 floating point representation:
i. Single Precision. (32 bits)
ii. Double Precision. (64 bits)

Using double precision, you can represent a larger or a smaller number than single precision.

Normalized Number

Binary Point (representing Binary Numbers)

⇒ A binary number is Normalized if :

i) Only one digit before the binary point.

ii) And that digit must be a non-zero number.

⇒ 11.00101×2^{35} ✗ not a normalized number.

⇒ 1.100101×2^{37} ✓ a normalized number.

* To normalize a number you need to shift the binary point (.) left or right until you have a single non-zero digit before the binary point.

⇒ If you shift left, the number of times you left shifted will be added with the exponent.

⇒ 110.111×2^{35}

⇒ $1.10111 \times 2^{35+2}$

⇒ If you shift right, the number of times you right shifted will be subtracted from the exponent.

⇒ 0.00110

⇒ 0.00110×2^{0}

⇒ $1.10 \times 2^{0-3} = 1.10 \times 2^{-3}$

IEEE-754 Floating Point Representation

	Sign Bit	Exponent	Fraction
Single P.	1 bit	8 bits	23 bits
Double P.	1 "	11 "	52 "

IEEE-754 Single Precision Format (32-bit)

(Biased)

Sign Bit	Exponent	Fraction
1	8	23

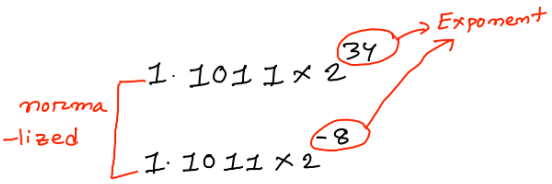
Sign Bit = 0 ⇒ positive number

1 ⇒ negative "

Exponent = It will be represented as unsigned number.

8 bit unsigned binary range = 0 to 2^8-1

= 0 to 255



But 0000 0000 and 1111 1111 are reserved, so the range for biased exponent is 1 to 254

If the size of biased exponent field is n bits, Bias = $2^{(n-1)}-1$

Hence, for 8 bit biased exponent, bias = 2^7-1 = 127

IEEE-754 Double Precision Format (64-bit)



Sign Bit = 0 ⇒ positive number

1 ⇒ negative "

Exponent = It will be represented as unsigned number.

11 bit unsigned binary range = 0 to $2^{11}-1$

= 0 to 2047

But. 000 0000 0000 and 111 1111 1111 are reserved, so the range for biased exponent is 1 to 2046

If the size of biased exponent field is n bits, Bias = $2^{(n-1)}-1$

Hence, for 11 bit biased exponent, bias = $2^{10}-1$ = 1023

Decimal to IEEE-754 Floating Point Conversion

Steps:

1. Convert the decimal number to binary number.
2. Normalize the binary number.
3. Find the biased exponent.
4. Sign Bit.
5. Find the fraction.
6. Encode accordingly.

Example:

Convert 50.6749 to IEEE-754 single precision floating point representation. Show your final answer in Hex format. Consider 10 bits while converting from decimal to binary.

Solution:

$50 = 11\ 0010$

$.6749 = 10\ 1011\ 0011\ldots$

(i) $50.6749 = (11\ 0010.10\ 1011\ 0011\ldots)$

$= 11\ 0010.10\ 1011\ 0011\ldots \times 2^0$

(ii) $= 1.1001\ 0101\ 0110\ 011\ldots \times 2^5$ ← actual exponent

(iii) Bias $= 2^{8-1} - 1 = 2^7 - 1 = 127$

$\therefore$ Biased exponent $= 5 + 127 = 132 = 1000\ 0100$

(iv) positive number; sign bit = 0

(v) $1.1001\ 0101\ 0110\ 011\ldots \times 2^5$

fraction

Fraction = 1001 0101 0110 011 00000000

rest of the bits will be filled up by 0s.

$.6749 \times 2 = 1.3498$

$.3498 \times 2 = 0.6996$

$.6996 \times 2 = 1.3992$

$.3992 \times 2 = 0.7984$

$.7984 \times 2 = 1.5968$

$.5968 \times 2 = 1.1936$

$.1936 \times 2 = 0.3872$

$.3872 \times 2 = 0.7744$

$.7744 \times 2 = 1.5488$

$.5488 \times 2 = 1.0976$

(Biased)

Sign Bit	Exponent	Fraction
0	1000 0100	1001 0101 0110 011 00000000

$50.6749 = 0100\ 0010\ 0100\ 1010\ 1011\ 0011\ 0000\ 0000$

$= 0x424AB300$

Example:

Convert -0.0232 to 12-bit IEEE-754 representation where the size of the exponent field is 4 bits. Show your final answer in Hex format.

Solution:

(i) $-0.0232 = -0.00000101$

(ii) $-0.00000101 = 1.01 \times 2^{-6}$ ← actual exponent; 1.01 ← fraction

(iii) $\text{Bias} = 2^{4-1} - 1 = 7$

$\therefore$ Biased Exponent $= -6 + 7 = 1 = 0001$

(iv) Sign Bit $= 1$.

(v) Fraction $= 000\ 0000$

1	0001	000 0000

$-0.000232 = 1000\ 1000\ 0000$

$= 0x880$ (Ans)

IEEE-754 Floating Point to Decimal Conversion

Steps:

1. Convert the Hex/Decimal number to binary number.
2. Arrange the binary number according to the given IEEE format.
3. Determine the sign.
4. Find out the actual exponent from the biased exponent field.
5. Convert Fraction to Decimal.
6. Final number = $(-1)^{Sign\ Bit} \times (1 + Fraction) \times 2^{Actual\ Exponent}$.

Example:

Convert the given IEEE-754 single precision floating point number 0xF2400120 to decimal.

Solution:

(i) 1111 0010 0100 0000 0000 0001 0010 0000

(ii) 1 111 00100 100 0000 0000 0001 0010 0000

1 — Sign Bit; 111 00100 — Biased Exp.; 100 0000 0000 0001 0010 0000 — Fraction

(iii) Sign = −

(iv) Biased exp. = 111 0010 0 = 228

$Bias = 2^{8-1} - 1 = 127$

$\therefore$ Exponent = 228 − 127 = 101

(v) Fraction = 100 0000 0000 0001 0010 0000

= 0.100 0000 0000 0001 0010 0000

= 0.5000343323

(vi) Decimal Value = $(-1)^1 \times (1 + 0.5000343323) \times 2^{101}$

$= -1.5000343323 \times 2^{101}$

Floating Point Addition/Subtraction

Given A and B are both floating-point numbers.
Steps:
1. Make sure both numbers are in Binary.
2. Normalize both A and B.
3. Align the binary point so that the lower exponent matches with the higher exponent.
4. Now add or sub accordingly.
5. Normalize the result.

Ex: $9.999 \times 10^{1} + 1.610 \times 10^{-1}$; size of exponent field is 3 bits

$= 99.99 + 0.1610$

$= 1100011.1111\ 1101\ 01 + 0.0010\ 1001\ 00$

$= 1.1000\ 1111\ 1111\ 0101 \times 2^{6} + 1.0100\ 100 \times 2^{-3}$

$= 1.1000\ 1111\ 1111\ 0101 \times 2^{6} + 0.0000\ 0000\ 1\ 0100\ 100 \times 2^{6}$

$= 1.10010 \times 2^{6}$ (Ans)

Bias $= 2^{3-1} - 1 = 3$

Biased Exp. $= 3 + 6 = 9$

Range $= 0$ to $2^{3}-1$

$= 0$ to 7

$= 1$ to 6 [reserved 0 and 7]

upper range

$9 > 6 \Rightarrow$ So, overflow

$1101\ 00.1110\ 11 \times 2^{8} + 1011\ 0.1111\ 1 \times 2^{7}$

$= 1.1010\ 0111\ 011 \times 2^{13} + 1.0110\ 1111\ 1 \times 2^{11}$

$= 1.1010\ 0111\ 011 \times 2^{13} + 0.0101\ 1011\ 111 \times 2^{13}$

$= 10.0000\ 0011\ 010 \times 2^{13}$

$= 1.0000\ 0001\ 1010 \times 2^{14}$ (Ans)

Floating Point Multiplication

Given A and B are both floating-point numbers.
Steps:
1. Make sure both numbers are in Binary.
2. Normalize both A and B.
3. Add the exponents.
4. Now multiply accordingly.
5. Normalize the result.
6. Determine the sign of the operation.

Ex: $1.110 \times 2^{5} \times 1.11 \times 2^{-5}$

$= 1.110 \times 1.11 \times 2^{5+(-5)}$

$= 11.0001 \times 2^{0}$

$= 1.10001 \times 2^{1}$ (Ans)

Overflow-Underflow detection for IEEE-754 format

Overflow / Underflow detection: → Given number is too small to represent using the mentioned system.

Step 1: Find the biased exponent of the answer.

Step 2: " " range of the biased exponent of the given system. (1 to upper Range)

Step 3: Detection:

```
if (Biased exponent < 1):
    underflow
else if (Biased exponent > upper Range)
    overflow
else : [1 ≤ Biased Exp ≤ upper Range]
    No over/under flow
```

Floating Point instructions in RISC-V

Suppose, two single Preci. floating point numbers A, B are stored in memory. The memory locations are directly stored in register X_{10}, X_{11}. Write necessary code to store the result of A+B in the memory address that is stored in X_{13}.

Soln:

```
flw     f1,  0(X10)   ; f1 = A
flw     f2,  0(X11)   ; f2 = B
fadd.s  f3,  f1, f2   ; f3 = A+B
fsw     f3,  0(X13)
```

Floating point instruction list:

Instruction	Syntax & Example	Operation	Note
fadd.s	fadd.s rd, rs1, rs2 Ex: fadd.s f_3, f_2, f_5	Perform single-precision floating-point addition rd = rs1 + rs2	For all the red marked instructions, if the numbers are in double precision then just add "d" instead of "s" Ex: fadd.d rd, rs1, rs2 Perform double-precision floating-point addition rd = rs1 + rs2
fsub.s	fsub.s rd, rs1, rs2 Ex: fsub.s f_3, f_2, f_5	Perform single-precision floating-point subtraction rd = rs1 - rs2	
fmul.s	fmul.s rd,rs1,rs2 Ex: fmul.s f_3, f_2, f_5	Perform single-precision floating-point multiplication. rd = rs1 * rs2	
fdiv.s	fdiv.s rd,rs1,rs2 Ex: fdiv.s f_3, f_2, f_5	Perform single-precision floating-point division. rd = rs1 / rs2	
fmadd.s	fmadd.s rd,rs1,rs2,rs3 Ex: fmadd.s f_3, f_2, f_5, f_6	Perform single-precision fused multiply addition. rd = (rs1 * rs2) + rs3	
fnmadd.s	fnmadd.s rd,rs1,rs2,rs3 Ex: fnmadd.s f_3, f_2, f_5, f_6	Perform negated single-precision fused multiply addition. rd = -(rs1 * rs2) - rs3	
fmsub.s	fmsub.s rd,rs1,rs2,rs3 Ex: fmsub.s f_3, f_2, f_5, f_6	Perform single-precision fused multiply subtraction. rd = (rs1 * rs2) - rs3	
fnmsub.s	fnmsub.s rd,rs1,rs2,rs3 Ex: fnmsub.s f_3, f_2, f_5, f_6	Perform negated single-precision fused multiply subtraction. rd = -(rs1 * rs2) + rs3	
fsqrt.s	fsqrt.s rd,rs1 Ex: fsqrt.s f_3, f_2	Perform single-precision square root. rd = sqrt(rs1)	
feq.s	feq.s rd,rs1,rs2 Ex: feq.s x_3, f_2, f_5	Performs a comparison between two floating point numbers. If both are equal stores 1 in rd else stores 0.	
flt.s	flt.s rd,rs1,rs2 Ex: flt.s x_3, f_2, f_5	Performs a comparison between two floating point numbers. If (rs1 < rs2) stores 1 in rd else stores 0.	
fle.s	fle.s rd,rs1,rs2 Ex: fle.s x_3, f_2, f_5	Performs a comparison between two floating point numbers. If (rs1 <= rs2) stores 1 in rd else stores 0.	

Instruction	Syntax & Example	Operation
flw	flw rd,offset(rs1) Ex: flw f_3, 8(x_5)	Load a single-precision floating-point value from memory into floating-point register rd.
fld	fld rd,offset(rs1) Ex: fld f_3, 16(x_5)	Load a double-precision floating-point value from memory into floating-point register rd.
fsw	fsw rs2,offset(rs1) Ex: fsw f_3, 8(x_5)	Store a single-precision value from floating-point register rs2 to memory.
fsd	fsd rs2,offset(rs1) Ex: fsd f_3, 16(x_5)	Store a double-precision value from floating-point register rs2 to memory.
fcvt.s.w	fcvt.s.w rd,rs1 Ex: fcvt.s.w f_3, x_5	Converts a 32-bit **signed** integer, in integer register rs1 into a floating-point number in floating-point register rd.
fcvt.w.s	fcvt.w.s rd,rs1 Ex: fcvt.w.s x_3, f_5	Convert a floating-point number in floating-point register rs1 to a signed 32-bit in integer register rd.
fmv.w.x	fmv.w.x rd,rs1 Ex: fmv.w.x f_3, x_5	Move the single-precision value encoded in IEEE 754 standard encoding from the lower 32 bits of integer register rs1 to the floating-point register rd.
fmv.x.w	fmv.x.w rd,rs1 Ex: fmv.x.w x_3, f_5	Move the single-precision value in floating-point register rs1 represented in IEEE 754 encoding to the lower 32 bits of integer register rd.

Formulas

1. $Bias = 2^{(n-1)} - 1\ ;\ n = Size\ of\ the\ exponent\ field$

2. $Biased\ Exponent = Bias + Actual\ Exponent$
   $Actual\ Exponent = Biased\ Exponent - Bias$

3. $Biased\ Exponent\ Range = 0\ to\ 2^n - 1\ ;\ [upper\ \&\ lower\ limit\ is\ reserved]$
   $= 1\ to\ 2^n - 2\ ;\ [Usable\ range]$

4. $Actual\ Exponent\ Range = (0 - Bias)\ to\ (2^n - 1 - Bias)$
   $= (1 - Bias)\ to\ (2^n - 2 - Bias)\ ;\ [Usable\ range]$

5. $Decimal\ Number = (-1)^{Sign\ Bit} \times (1 + Fraction) \times 2^{Actual\ Exponent}$